

WEEK 5 · DAY 2

Agent Collaboration & Multi-Agent Orchestration

How Specialized Agents Hand Off Work, Combine Results, and Stay Auditable Together

1-Hour Live Demonstration & Architecture Discussion

Rajesh Pasham

39 Slides · 4 Topic Groups

Program Context

How today's session fits the wider 8-week delivery

Session Type	Live demonstration and architecture discussion — building directly on Day 1's single-agent foundations
Platform	Individual Palantir Developer Tier accounts, in continuous use since Week 1, Day 3
This Week's Lab (Day 3)	Multi-Agent Insurance Workflow — orchestrated agents that collaborate, invoke Actions and Functions, and write back to your Ontology
Skill Gained This Week	AI Agent Engineer — the fifth rung of the program's Executive Skills Matrix

Where Today Fits

From one agent reasoning alone, to specialized agents working together

Day 1 gave you everything one agent needs: planning, memory, tool invocation, and the guardrails that bound it. But Wednesday's lab doesn't build one agent — it builds two per batch, working together on a single goal. Today is about the mechanics that make that possible: how agents hand off work, how an Orchestrator decides who does what, and how the results of several agents combine into one trustworthy outcome.

Not new theory in isolation — it is Day 1's architecture, multiplied.

Every agent in today's patterns still plans, remembers, and invokes tools exactly as Day 1 described. What's new is the layer above a single agent: how its output becomes another agent's input, and who decides the order.

Today's Roadmap

Four topic groups, building toward Wednesday's multi-agent lab

1

Agent Collaboration Fundamentals

Why one agent isn't enough, and the basic patterns for how agents work together

2

The Orchestrator Pattern

How an Orchestrator Agent routes work and combines results, walked through on a full claim

3

Multi-Agent System Design

State management, latency and cost, deadlock risk, and observability across agents

4

Trust, Quality & Your Batch's Orchestration

Evaluating and auditing multi-agent outcomes, and how your batch's two agents will collaborate on Day 3

By the End of Today

The vocabulary Wednesday's multi-agent lab assumes

- ✓ **Explain why a single agent's scope should stay narrow, and what that implies for collaboration**

The Day 1 case for specialization now needs a mechanism to actually connect specialists

- ✓ **Describe agent-as-tool, sequential handoff, and supervisor-worker collaboration patterns**

Three different shapes of collaboration, suited to different goals

- ✓ **Explain what an Orchestrator Agent actually does**

Decompose a goal, route to the right specialist, and combine what comes back

By the End of Today

The vocabulary Wednesday's multi-agent lab assumes (cont'd)

- ✓ **Identify the state, latency, and deadlock risks specific to multi-agent runs**

Problems that don't exist for a single agent, but do the moment two agents depend on each other

- ✓ **Describe how a multi-agent run stays auditable**

Which agent decided what, and why — extending Week 3's writeback governance to a chain of agents

- ✓ **Describe your batch's own orchestration pattern for Wednesday's lab**

Which of your batch's two agents leads, and how the handoff between them is triggered

TOPIC 1

Agent Collaboration Fundamentals

Why one agent isn't enough, and the basic shapes collaboration takes

Why a Single Agent Isn't Enough

Day 1's case for narrow scope now needs a way to connect the specialists it created

A real goal often spans more than one responsibility

“Assess this claim end to end” touches coverage, fraud risk, and settlement — three responsibilities Day 1 deliberately split across three narrow agents.

One wide agent recreates the problem specialization solved

Folding claims, fraud, and settlement logic back into a single agent to avoid coordination brings back exactly the evaluation, security, and deployment costs Day 1's Topic 4 argued against.

Collaboration is the missing piece, not a new agent design

Every agent in today's patterns still plans, remembers, and calls tools exactly as Day 1 described — what's new is how one agent's output reaches another.

Three Shapes of Collaboration

Different goals call for different collaboration structures — Wednesday's lab uses more than one

Pattern	How It Works	Best Suited To
Agent-as-tool	One agent is exposed to another simply as a callable tool, the same way a Function is	A narrow, well-defined sub-task — the Underwriting Agent calling the Risk Assessment Agent for a fresh score
Sequential handoff	Agent A completes its part, then explicitly hands the goal and its findings to Agent B	A clear stage boundary — Claims Agent finishing coverage validation before Fraud Agent begins
Supervisor-worker	An Orchestrator Agent decomposes the goal and assigns pieces to several worker agents, then combines results	Goals that need more than one specialist's input to reach a single combined decision

Agent-as-Tool: The Simplest Collaboration Pattern

No new mechanism required — an agent is just another tool with a description

An agent's goal-taking interface looks like a Function's input schema

The calling agent doesn't need to know how the called agent reasons internally — only what goal to hand it and what shape of result to expect back.

This is exactly how the Underwriting Agent uses the Risk Assessment Agent

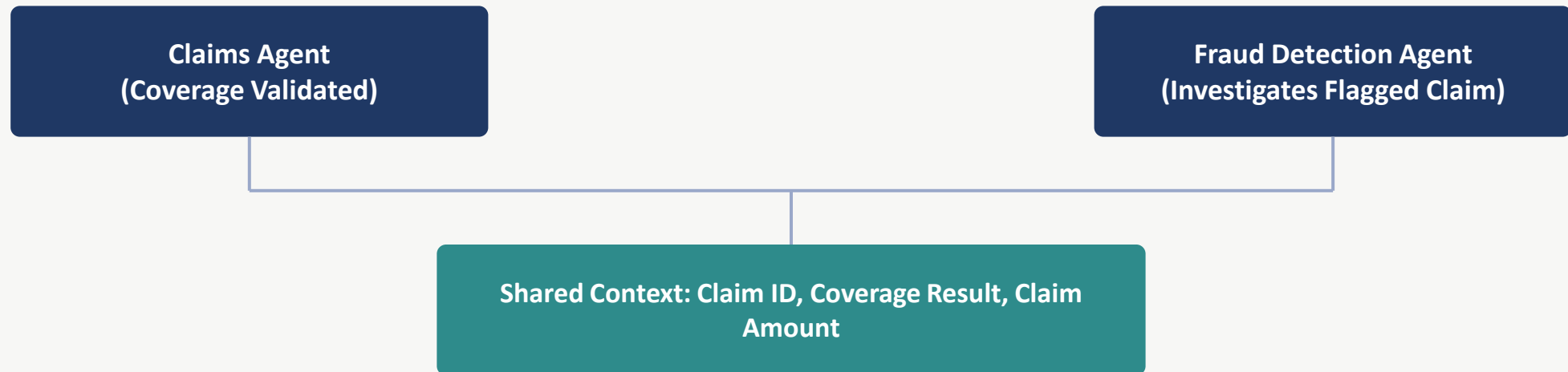
Rather than recomputing a risk score itself, the Underwriting Agent calls the Risk Assessment Agent as a tool and waits for a fresh score, the same way it would call any Week 3 Function.

Nesting is possible, but bounded

An agent-as-tool can itself call further tools — but an unbounded chain of agents calling agents is exactly the runaway-scope risk Day 1's guardrails topic warned about.

Sequential Handoff, Concretely

One agent's completed findings become the next agent's starting context



The Fraud Detection Agent doesn't start from nothing — it receives exactly the shared context the Claims Agent decided was relevant, not the Claims Agent's full internal reasoning or unrelated memory.

Shared Context: What Actually Gets Passed Between Agents

A deliberate, scoped handoff — not a shared memory pool every agent can read

The goal, and what's been established so far

“Investigate FNOL-2201 for fraud”, plus the coverage result already confirmed — the receiving agent doesn't repeat work the sending agent already did.

Only the relevant slice of short-term memory

Day 1's short-term memory scoping still applies here — the Fraud Detection Agent gets the claim details it needs, not the Claims Agent's entire working memory for every claim it has ever touched.

A reference back to the originating agent and step

So an audit trail can reconstruct the full chain later — this is what Topic 3's observability discussion depends on.

Conflict Resolution: When Two Agents Disagree

Collaboration has to define what happens when specialists reach different conclusions

A real example: coverage says pay, fraud says hold

The Claims Agent's coverage validation clears the claim for settlement; the Fraud Detection Agent flags the same claim as a suspicious pattern match.

Disagreement is a signal to escalate, not a tie to break silently

The Orchestrator does not pick whichever agent answered first, or average the two outcomes — a flagged conflict routes to the Compliance Agent or a human, per your batch's governance plan.

The conflict itself is worth recording

That two specialized agents disagreed is useful information for Week 6's Evals — a pattern of frequent disagreement between two agents may mean one of their tools or prompts needs revision.

TOPIC 2

The Orchestrator Pattern

Decompose, route, and combine — walked through on a full claim

What an Orchestrator Agent Actually Does

Three jobs, none of which involve doing the specialist work itself

Decompose the goal

Break “assess this claim end to end” into the sub-goals each specialist agent actually handles — coverage validation, fraud review, settlement recommendation.

Route each sub-goal to the right agent

Decide which specialized agent is invoked for which piece, and in what order — sequential where one result gates the next, parallel where it doesn't.

Combine what comes back into one outcome

Synthesize the specialists' results into a single recommendation or writeback, resolving conflicts per the rules Topic 1 introduced rather than silently favoring one agent.

Routing Logic: Two Ways an Orchestrator Decides

Neither is universally better — the choice depends on how predictable your routing rules are

Approach	How It Decides	Trade-off
Rules-based routing	A fixed condition (claim amount, product line, prior flags) determines which agent is invoked next	Fast, predictable, and easy to audit — but only handles the cases its author anticipated
LLM-based routing	The Orchestrator itself reasons over the goal and available agents to decide who to invoke	Handles novel or ambiguous cases — but is harder to predict and needs its own evaluation, per Week 6

Sequential vs. Parallel Execution

Whether specialists run one after another, or at the same time, changes both speed and dependency

Execution Mode	How It Runs	When to Use It
Sequential	Each agent runs only after the previous one completes and hands off its result	When one agent's output is required input for the next — coverage before settlement
Parallel	Independent agents run at the same time, and the Orchestrator waits for all to complete before combining	When agents don't depend on each other's output — risk scoring and document classification on the same application

Combining Results: Aggregation and Synthesis

The Orchestrator's final job is turning several agents' outputs into one coherent outcome

Structured aggregation

Each agent returns a defined output shape (Week 2's Output Struct pattern, extended) — the Orchestrator merges these fields into one combined record rather than free-form text.

Synthesis for anything that isn't purely structured

Where agents disagree or provide qualitative reasoning, the Orchestrator (or a human, per the conflict-resolution rule) produces the final narrative recommendation.

The combined output is what gets proposed as an Action

Only after aggregation does the result reach Week 3's Ontology Actions — the writeback still represents one governed decision, not several competing ones.

The Orchestrator's Own Tools

An Orchestrator Agent is still an agent — it plans, remembers, and invokes tools, just at one level up

A decomposition capability, not a fixed script

The Orchestrator's own planning step (Day 1's Topic 2) is what breaks “assess this claim” into sub-goals — it is not a hardcoded if/then list of which agent runs when.

Specialist agents themselves are its tools

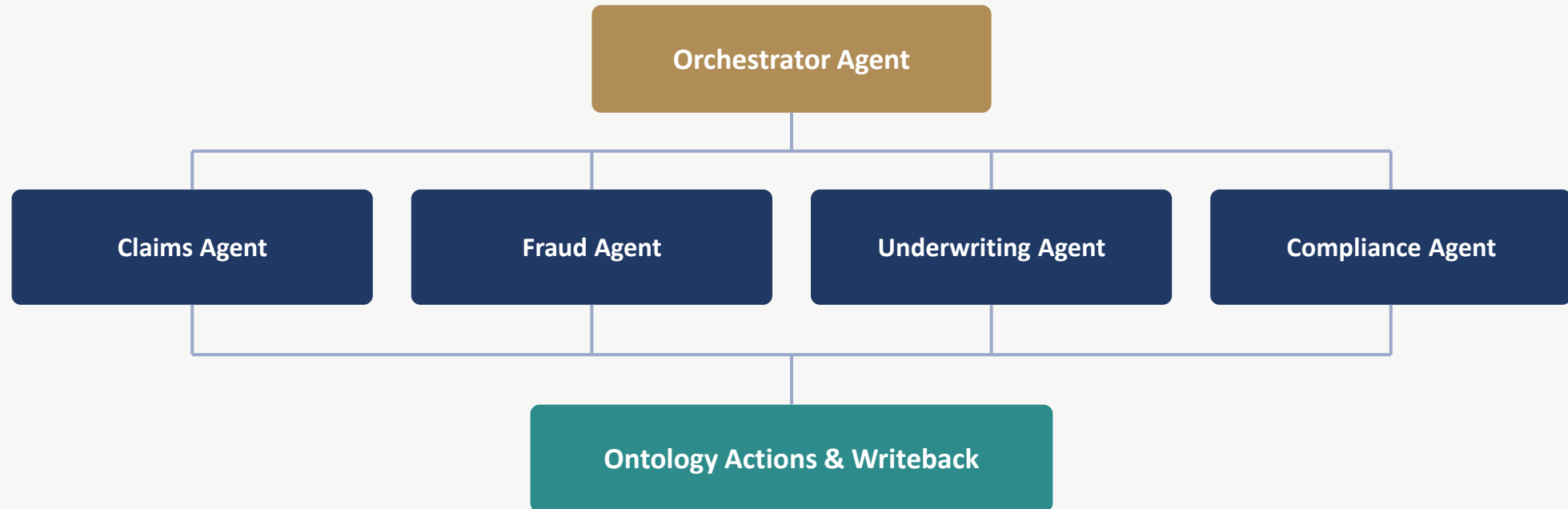
Where a Claims Agent's tools are Functions and Actions, the Orchestrator's tools are the Claims Agent, the Fraud Detection Agent, and the others — agent-as-tool, applied at the top level.

Its own memory tracks the run so far, not claim history

The Orchestrator's short-term memory holds which sub-goals are done, which are pending, and what each specialist returned — exactly the state a supervisor needs to combine results correctly.

The Orchestrator Pattern, in Full

Decompose → route to specialists → combine → one governed writeback



This is the same diagram from Day 1's preview — today gives you the actual decision logic inside the top box: how it decomposes, how it routes, and how it reconciles what comes back before anything reaches the writeback path.

Walking a Full Claim Through Orchestration

FNOL-2201, from intake to a governed writeback — four agents, one Orchestrator

Orchestrator receives the goal: “Assess FNOL-2201 end to end”

1

It decomposes this into three sub-goals: validate coverage, assess fraud risk, recommend settlement.

Sequential step: Claims Agent validates coverage first

2

Coverage is a gating fact — nothing downstream matters if the policy isn't active, so this step runs before anything else.

Parallel step: Fraud Detection Agent and a document check run together

3

Neither depends on the other's output, so the Orchestrator runs them concurrently to save time, per Topic 2's parallel-execution guidance.

Walking a Full Claim Through Orchestration

FNOL-2201, from intake to a governed writeback — four agents, one Orchestrator (cont'd)

Fraud Detection Agent flags a pattern; Claims Agent's draft recommendation said approve

4

This is the conflict scenario from Topic 1 — the Orchestrator does not resolve it silently.

Orchestrator routes the conflict to the Compliance Agent

5

Per your batch's governance plan, a coverage-vs-fraud conflict is exactly the case Compliance exists to adjudicate.

Combined result reaches a human for the final Action submission

6

The Orchestrator's synthesized recommendation, with the conflict and Compliance's read on it, becomes the case a human actually approves.

TOPIC 3

Multi-Agent System Design

State, latency, deadlock, and observability across agents

Supervisor vs. Decentralized Collaboration

Where decision authority sits changes how the whole system behaves

Model	Who Decides	Governance Implication
Supervisor (Orchestrator-led)	One Orchestrator Agent holds routing and combination authority	A single point to secure, audit, and evaluate — the pattern this program builds toward
Decentralized (peer-to-peer)	Agents invoke each other directly, with no central coordinator	Harder to audit and secure — every agent-to-agent edge needs its own governance, which is why Wednesday's lab uses a supervisor model

State Management Across Agents

Two agents working on the same claim can't each hold their own uncoordinated copy of the truth

Shared state lives in the Ontology, not in an agent's private memory

The Claim object itself, not a copy inside the Claims Agent's working memory, is the source of truth every collaborating agent reads from.

Each agent's own memory stays scoped to its own reasoning

The Fraud Detection Agent's short-term memory holds its own investigation notes — it doesn't need, and shouldn't hold, the Claims Agent's internal reasoning steps.

A race condition is possible when two agents write concurrently

If the Fraud Detection Agent and the Claims Agent could both propose an update to the same Claim object at once, the same Action-submission discipline from Week 3 — one committed writeback at a time — prevents a corrupted result.

Latency and Cost of Multi-Agent Runs

Every hop between agents is an additional reasoning step, and additional cost

Sequential hops add up

Four agents each taking a few seconds to plan and act means a claim assessment that could take well over a minute end to end — a cost invisible when you tested one agent alone on Day 1.

Parallel execution is a genuine lever, not just an optimization

The parallel step in today's walkthrough exists specifically because fraud review and document classification don't depend on each other — running them concurrently is a real latency reduction, not a nice-to-have.

Cost scales with the number of agent invocations, not just tool calls

Each agent's own reasoning is itself an LLM call — a supervisor-worker pattern with five agents costs meaningfully more per run than a single agent, a trade-off Week 6's cost-management topic returns to.

Deadlock and Loop Risks

The one failure mode that doesn't exist until agents can trigger each other

A cycle is a design defect, not a runtime fluke

If the Fraud Detection Agent's output can trigger the Claims Agent, and the Claims Agent's output can trigger the Fraud Detection Agent again, a legitimate disagreement can loop indefinitely.

Bounded hop counts prevent runaway chains

The same run-length guardrail from Day 1 applies at the multi-agent level — an Orchestrator should cap how many handoffs a single goal can trigger before forcing escalation.

Trace every trigger path before Wednesday, not after

Before wiring your batch's two agents together in Agent Studio, map out every condition under which each agent could invoke the other, and check for a cycle on paper first.

Designing for Observability Across Agents

A multi-agent run has to be traceable as one story, not four disconnected logs

Every hop carries its originating context forward

The reference back to the sending agent and step, introduced in Topic 1's shared-context discussion, is what lets a single run be reconstructed end to end.

A trace should read like the scenario walkthrough above

Goal received, decomposed, routed, one conflict, one resolution, one final writeback — an auditor should be able to follow exactly that sequence from the logs alone.

This is Week 6's Evals and observability, arriving early

Multi-agent systems are precisely where LLMOps discipline stops being optional — Week 6 formalizes the tracing this topic has been assuming you'll want.

TOPIC 4

Trust, Quality & Your Batch's Orchestration

Evaluating multi-agent outcomes, and building your batch's pattern

Evaluating Multi-Agent Outcomes

The same evaluation discipline from Week 2, now aimed at a chain of agents instead of one

Evaluate the combined outcome, not just each agent alone

A Claims Agent and a Fraud Detection Agent can each individually score well on their own narrow task, while the combined orchestration still reaches the wrong final recommendation.

Track how often agents disagree, and how conflicts resolve

A rising disagreement rate between two specific agents is itself a signal worth investigating — it may point to a stale tool, a bad prompt, or a genuine edge case neither agent handles well.

This becomes the Weekly Evaluation's “Agent Review”

Your program's Weekly Evaluation Framework specifically calls out Agent Review for this week — this is what that review actually looks at.

Auditability: Which Agent Did What

A governed writeback has to survive the question “why did the system do this?”

Every Action submission traces to a decision chain, not just a final result

The Object Explorer submission from Week 3 now carries a reference to the full orchestration: which agents ran, in what order, and what each contributed.

A human reviewing an outcome should see the same walkthrough this session gave you

Goal, decomposition, routing, any conflict, and resolution — presented clearly enough that a non-technical reviewer can follow it.

This is not optional for regulated writebacks

An insurance settlement decision made by a multi-agent system needs the same auditability a human decision-maker's reasoning would be expected to have.

Citation Across Agents

Extending Week 2's groundedReferences to a chain of agents, not just a chain of documents

Each agent's contribution carries its own source

The Fraud Detection Agent's flag cites the specific prior claims or pattern it matched against, the same way Week 4's document intelligence cited a specific chunk.

The combined outcome cites each contributing agent, not just the Orchestrator

A claims handler reviewing the final recommendation should be able to see which specific agent raised which specific concern, not just a single opaque “AI recommendation.”

This is what makes escalation useful rather than just noisy

A human resolving the conflict from today's walkthrough needs to see exactly what the Claims Agent and the Fraud Detection Agent each based their conclusion on.

Why This Matters for Day 3's Lab

Every concept today becomes a configuration decision in Wednesday's Agent Studio session

You will configure a supervisor pattern, not a decentralized one

Your batch's two agents connect through an Orchestrator, following Topic 3's governance argument for why this program uses that model.

You will decide sequential vs. parallel for your own two agents

Based on whether your second agent's work depends on your first agent's output, exactly as today's routing-logic discussion framed the choice.

You will define what happens on disagreement, before you need it

Deciding your batch's conflict-resolution rule on paper first avoids discovering the gap live, during the lab.

Your Batch's Orchestration Pattern for Day 3

Which agent leads, how the handoff triggers, and what execution mode applies

Batch	Leading Agent → Second Agent	Handoff Trigger	Execution Mode
Batch 1 — India	Claims Agent → Fraud Detection Agent	Claim amount or pattern crosses your batch's threshold	Sequential — fraud review follows coverage validation
Batch 2 — India	Underwriting Agent → Risk Assessment Agent	A decision requires a current, not stale, risk score	Sequential — decision waits on the fresh score
Batch 3 — Europe	Fraud Detection Agent → Compliance Agent	A flagged claim requires regulatory sign-off before referral	Sequential — compliance gates the referral
Batch 4 — Europe	Customer Support Agent → Policy Recommendation Agent	A query exceeds support scope and needs a coverage or renewal recommendation	Sequential — recommendation follows the handoff

Pre-Flight Checklist Before Wednesday's Lab

Five decisions worth settling on paper before you open Agent Studio

Which of your batch's two agents leads?

Decided in the table above — confirm it matches how your use case actually flows, not just the default assignment.

What triggers the handoff between them?

A threshold, a status change, or a query type — write the exact condition down, the same precision Week 1's Marking rules demanded.

Sequential or parallel?

Only parallel if the second agent's work never depends on the first agent's output — when in doubt, start sequential.

Pre-Flight Checklist Before Wednesday's Lab

Five decisions worth settling on paper before you open Agent Studio (cont'd)

What happens on disagreement?

Escalate to a third agent, or to a human — decide this before your first test run surfaces a real conflict.

Where does the trail live?

Confirm every hop between your two agents carries enough shared context back to reconstruct the run afterward.

Common Pitfalls

Patterns worth avoiding before Wednesday's lab



Letting every agent talk to every other agent

Unrestricted agent-to-agent access is the multi-agent equivalent of skipping RBAC — route collaboration through a defined Orchestrator or handoff contract, not an open mesh.



No timeout on a handoff

An agent waiting indefinitely for another agent's result can stall an entire run — Wednesday's lab requires every handoff to have a bounded wait.



Two agents that can trigger each other in a cycle

A Fraud Agent that always re-triggers the Claims Agent, which re-triggers the Fraud Agent, is a deadlock waiting to happen — trace every trigger path before Wednesday for cycles.

Common Pitfalls

Patterns worth avoiding before Wednesday's lab (cont'd)

Combining results without resolving disagreement

If the Claims Agent recommends approval and the Fraud Agent recommends a hold, silently picking one is worse than surfacing the conflict to a human.

Treating the Orchestrator as exempt from governance

An Orchestrator Agent still needs RBAC/ABAC on what it can route to and what it can see — orchestration is not a loophole around Week 1's access model.

Losing the audit trail across a handoff

If Agent A's reasoning isn't visible once Agent B takes over, no one can reconstruct why the combined decision was made — every hop needs to stay in the log.

Recap & What's Next

What We Covered Today

- Why single-agent scope stays narrow, and the patterns that connect specialists
 - Agent-as-tool, sequential handoff, and supervisor-worker collaboration
 - What an Orchestrator Agent does — decompose, route, and combine
 - A full claim walked end to end across four collaborating agents
 - State, latency, cost, and deadlock risks unique to multi-agent runs
 - Auditability and evaluation for outcomes that span several agents
 - Your batch's own orchestration pattern for Wednesday's lab
-

Day 3 — Hands-On Lab: Multi-Agent Insurance Workflow